

Sistemas Distribuidos

Ejercicio Evaluable 1: Colas de Mensajes



Javier Moyano San Bruno - 100495884 - [100495884 @alumnos.uc3m.es](mailto:100495884@alumnos.uc3m.es)

Héctor Herraiz Díez - 100499734 - 100499734@alumnos.uc3m.es

Grupo 82 - Ingeniería informática

1. Introducción

Este proyecto implementa un **sistema distribuido** para la gestión de tuplas de la forma <key, value1, value2, value3>. Se basa en un modelo **cliente-servidor** donde los clientes interactúan con un **servidor centralizado** mediante **colas de mensajes POSIX**.

Cada tupla consta de:

- **key**: un número entero que actúa como identificador único.
- **value1**: una cadena de caracteres de hasta 255 caracteres.
- **value2**: un vector de valores double, con una longitud N_value2 de entre 1 y 32 elementos.
- **value3**: una estructura Coord con dos valores enteros {x, y}.

El servidor se encarga de gestionar el almacenamiento y procesamiento de estas tuplas, mientras que los clientes utilizan una biblioteca dinámica (libclaves.so) para acceder a los servicios de manera transparente.

2. Arquitectura del Sistema

El sistema sigue un modelo de **cliente-servidor distribuido**. Se compone de los siguientes elementos:

- **Servidor (servidor-mq.c)**:
 - Gestiona las tuplas en memoria.
 - Atiende solicitudes de los clientes de manera concurrente mediante hilos.
 - Se comunica con los clientes mediante **colas de mensajes POSIX**.
 - **Proxy del cliente (proxy-mq.c)**:
 - Implementa las funciones de la API (destroy, set_value, get_value, modify_value, delete_key, exist).
 - Se encarga de enviar peticiones al servidor y recibir respuestas mediante colas de mensajes.
 - **Biblioteca dinámica (libclaves.so)**:
 - Contiene la implementación del proxy para que los clientes usen la API sin interactuar directamente con las colas de mensajes.
 - **Cliente (app-cliente.c)**:
 - Programa que usa la API (libclaves.so) para realizar operaciones sobre las tuplas sin manejar la comunicación con el servidor directamente.
-

3. Comunicación Cliente-Servidor

La comunicación se realiza a través de **colas de mensajes POSIX**.

1. **El cliente crea una cola privada** con un nombre único.
2. **El cliente envía una solicitud** al servidor a través de una cola común.
3. **El servidor recibe la solicitud**, la procesa y envía la respuesta a la cola privada del cliente.
4. **El cliente espera la respuesta**, la recibe y cierra la cola.

El servidor maneja cada solicitud en un hilo independiente para procesarlas de manera concurrente y responder de forma eficiente a múltiples clientes.

4. Manejo de la Concurrency

Dado que el servidor debe atender múltiples clientes simultáneamente, se ha implementado un modelo basado en hilos (pthread). Cada vez que se recibe una solicitud de un cliente, se genera un nuevo hilo que se encarga de procesar la petición sin bloquear el resto del sistema.

Para evitar problemas de acceso concurrente a las estructuras de datos que almacenan las tuplas, se utilizan mutex (pthread_mutex_t). Esto garantiza que solo un hilo a la vez puede modificar la lista de tuplas, evitando condiciones de carrera y corrupción de datos.

Además, se han empleado variables de condición (pthread_cond_t) para coordinar la sincronización entre hilos y optimizar el rendimiento del sistema, reduciendo bloqueos innecesarios.

En el proxy del cliente, se ha añadido un mutex en la función send_request para evitar que múltiples hilos en el cliente puedan interferir entre sí al enviar y recibir mensajes de la cola. Esto asegura que las solicitudes se gestionen de forma segura y ordenada.

5. Implementación

El sistema se ha desarrollado en C usando **colas de mensajes POSIX** para la comunicación.

- **servidor-mq.c**: encargado de recibir peticiones de los clientes y procesarlas utilizando hilos.
- **proxy-mq.c**: envía las solicitudes al servidor en lugar de ejecutarlas directamente.
- **libclaves.so**: biblioteca compartida que encapsula proxy-mq.c para que los clientes puedan utilizar la API sin preocuparse de la comunicación.
- **app-cliente.c**: implementa un cliente de prueba que usa la API de libclaves.so.

6. Compilación y Ejecución

Compilación

El **Makefile** permite compilar todos los componentes del sistema con el comando *make*, generando los siguientes archivos:

- servidor → Ejecutable del servidor
- cliente → Ejecutable del cliente
- libclaves.so → Biblioteca dinámica utilizada por el cliente

Ejecución

1. Iniciar el servidor ejecutando *./servidor*.
 2. Ejecutar clientes en distintas terminales para probar la concurrencia con *./cliente*.
-

7. Plan de Pruebas

Se han diseñado diversas pruebas para validar el correcto funcionamiento del sistema:

7.1 Pruebas de Operaciones Básicas

- Inserción, consulta, modificación y eliminación de tuplas.
- Comprobación de existencia y eliminación masiva.

7.2 Pruebas de Manejo de Errores

- Intentos de inserción de claves duplicadas.
- Acceso a claves inexistentes.
- Introducción de valores fuera de rango.

7.3 Pruebas de Concurrencia

Se ejecutaron múltiples clientes simultáneamente para verificar que el servidor maneja correctamente el acceso concurrente.

7.4 Prueba de Límite de Cola de Mensajes

Se realizaron pruebas con un alto volumen de tuplas para comprobar la estabilidad del sistema.

8. Conclusión

El sistema desarrollado permite la gestión distribuida de tuplas mediante una arquitectura cliente-servidor basada en colas de mensajes POSIX. La implementación cumple con los requisitos de concurrencia y comunicación eficiente, permitiendo que múltiples clientes interactúen con el servidor sin interferencias. Se realizaron diversas pruebas que confirmaron el correcto funcionamiento del servicio en diferentes escenarios.